Illini Portal Fit Engine

A transfer-portal recruiting target board for Illinois Men's Basketball

Author: Mannat Talati · **Built for:** Illinois Men's Basketball Analytics Internship **Live app:** *(Streamlit link — see README)* · **Code:** *(GitHub link — see README)*

---

1. The problem

Roster construction in college basketball now runs through the transfer portal. Every spring, 1,500+ Division I players enter, and a staff has a few weeks to decide who to pursue. Illinois under Brad Underwood has been one of the most aggressive portal and international programs in the country, so this is not a side project for the staff — it *is* the roster.

The hard part is not finding *good* players; it is finding good players who **(a)** fill a **specific hole** the roster just opened, **(b)** fit how Illinois actually plays, and **(c)** are **realistic** to land. Doing that by eye across all of Division I is slow and biased toward the names you already know.

This tool turns that triage into a ranked, explainable board. It profiles Illinois's roster, detects what's leaving, and scores every eligible D-I player on a transparent **Fit Score** so the staff can start the portal window with a shortlist instead of a blank page.

2. The data — and how I sourced it

Everything is **public BartTorvik data**, pulled live over plain HTTP (no scraping tricks, no paywalled sources like KenPom):

Source	Endpoint	What it gives
Player advanced stats	`getadvstats.php?year=YYYY&csv=1`	5,000 players/season, 67 columns: archetype role, usage, efficiency, shooting splits, BPM/OBPM/DBPM, rebounding, steals/blocks, class, height, hometown
Team ratings	`YYYY_team_results.json`	365 teams: adjusted offense/defense, Barthag power rating, tempo

The player feed ships with **no header row**, so I reverse-engineered the column layout and then **validated it against the data itself** before trusting a single number:

- shooting splits reconcile (FTM ÷ FTA = FT%, etc.),
- `BPM == OBPM + DBPM` holds for **100%** of rows,
- `total rebounds == offensive + defensive` per game holds for **100%** of rows.

Columns I could not verify are named `extra_NN` and are **deliberately unused**, so every figure the tool reports is one I can explain on a whiteboard.

A real snapshot is committed to the repo, so the app loads instantly for a reviewer and the results are reproducible; a "Refresh from BartTorvik" button re-pulls live data on demand.

3. How the Fit Score works

For the current (2025-26) season, the engine reads Illinois as: **28-9, Barthag #6, the

2 offense in the country (adjOE 131.8), #20 defense, and — notably — the #289 tempo.**

That last number matters: Illinois is an *elite half-court* team, **not** a run-and-gun team, so the model rewards skill and shot-making over raw transition athleticism. The profile is *derived from data*, not assumed.

It then assumes seniors graduate (editable in the app) — for 2025-26 that's lead guard **Kylan Boswell**, stretch big **Ben Humrichous**, and **AJ Redd** — and measures what walks out the door: a chunk of the team's **playmaking, perimeter defense, and outside shooting**.

Each eligible candidate (non-Illinois, has remaining eligibility, cleared minimum minutes/games) gets a **0-100 Fit Score**, a weighted blend of four parts that are each shown separately:

1. **Production** — proven value: BPM, porpag, minutes load.
2. **Role fit** — does their position fill a *depleted* Illinois spot (guard / wing / big)?
3. **System fit** — do their strengths match the **specific** skills Illinois must replace? The weights here come straight from the departure analysis (e.g. "we lost 24% of our made threes" → shooting is weighted up).
4. **Attainability** — a realism lens from the player's current team strength, surfaced as a plain label: *High-major proven*, *Mid-major riser*, *Low-major standout*.

Every component is percentile-ranked **within the candidate pool**, so a score answers the question a coach actually asks: *"compared to the other realistic options, how good is this?"* All four weights, the departure list, and the need weights are **tunable live** in the app.

Example output (default Illinois needs, 2025-26):

Fit	Player	Team	Role	Why
86.8	Jason Rivera-Torres	Monmouth	Wing F	3.3% steal rate; 1.5 3PM/g; +5.5 BPM — perimeter D + shooting; mid-major riser
85.3	Tyler Tanner	Vanderbilt	Scoring PG	5.1 ast/g (29 AST%); +10.2 BPM — playmaking; high-major proven
85.1	Mason Falslev	Utah St.	Combo G	3.6% steal rate; 3.1 ast/g; +8.8 BPM — perimeter D + playmaking

Each row carries an auto-generated scouting note, and the **Player Detail** view breaks the score into its four bars next to the player's full statistical profile.

4. Three tools that make the board a scouting product

A ranked list is a start; a staff also needs to know *what the numbers mean at our level*, *who a player reminds us of*, and have something they can **print and hand to a coach**. Three additions do that — and the first two are calibrated on real history, not assumed.

4a. Big Ten Translation — what a box line becomes at Illinois's level

A 20-and-8 against low-major defenses is not a 20-and-8 in the Big Ten, because box-score counting stats are **not** opponent-adjusted. To make gaudy lines honest, I learned the deflation from data: I matched the **same player across consecutive seasons** using BartTorvik's stable player id, found everyone who **changed competition level**, and measured how much of each stat survived the jump. The calibration set is **990 real cross-season transfers**.

For each stat I fit a one-parameter retention line, anchored so a zero-gap move means no change (`ratio = 1 + slope × level_gap`), where the gap is the Barthag distance between the player's team and Illinois (Barthag **0.968**). The result is exactly the scouting lesson you'd hope for:

Stat	Survives a jump?	Keeps, at a typical mid-major→Illinois jump
Points / g	deflates most	~78%
Rebounds / g	deflates	~84%
Assists / g	deflates	~86%
Usage%	deflates mildly	~88%
Made 3s / g	travels well	~90%
TS% (efficiency)	**holds (slope ≈ 0)**	**~101%**

So **efficiency travels and made-shooting travels; raw volume deflates** — and the deflation scales with how big a leap the player is making. Concretely, a 21.4 ppg scorer in the SWAC projects to **~6.5 ppg** in the Big Ten ("major jump, high variance"), while a high-major star at BYU keeps ~93% of his scoring. The board shows each candidate's **raw line next to its projection**, a **Keep%**, and a **level-jump risk label** so a 25-ppg low-major name can't masquerade as a 25-ppg Big Ten name.

> Note on double-counting: the Fit Score's *production* component already uses opponent-**adjusted** inputs (BPM, adjusted ratings), so the translation model intentionally leaves the score alone and instead translates the **raw box stats coaches read directly** — keeping both numbers honest and separate.

4b. Player comps — "who does he play like?"

During portal season a staff constantly asks two questions: *a starter just left — which available transfers replace his style?* and *we like this target — who does he remind us of?* Both are nearest-neighbour problems in a **style space**: each player becomes a vector of role/skill rates (usage, shot diet, playmaking, ball security, rebounding, rim protection, perimeter activity, size, efficiency), **z-scored against the D-I rotation baseline** so every axis is comparable. Distance is plain Euclidean — no learned weights — reported as a 0-100 similarity. The **Program & Needs** page turns a departing Illini into a ranked list of transfer-eligible look-alikes; every target carries its closest high-major comps.

4c. Visual scouting cards + printable PDF

Coaches read cards, not 60-column CSVs. Each target renders a one-page card: a **percentile radar vs the Big Ten rotation** (every spoke is "how would this skill rank in our league?"), the **raw → Big Ten projected** box line, a **shot diet** (rim / mid / three share and accuracy), and the player's **comps** — exportable to a **self-contained 1-page PDF** the staff can print for the board.

5. How I built it

- **Language:** Python 3.
- **Data / model:** `pandas`, `numpy` — a small, readable package (`illini_fit/`) split into `schema` (validated column map), `fetch` (cache + live refresh), `profile` (team + roster), `needs` (departure-driven gap detection), `fit_score` (the scoring model), `translation` (the Big Ten retention model), `similarity` (the style-comp engine), and `scouting` (radar + PDF cards).
- **App:** `streamlit` — an interactive, Illini-themed web app (filters, weight sliders, ranked board with projected lines, player detail, scouting cards, CSV + PDF export); `matplotlib` for the radar and `xhtml2pdf` for the printable card.
- **Quality:** every module ships with self-checks run as `python -m illini_fit.<module>` — data row-count/Illinois-parse assertions; a test that a shooting-weighted board surfaces more shooters and scores stay in 0-100; that scoring deflates and efficiency holds in the translation model; that a player is his own closest comp. The app is verified headlessly with Streamlit's `AppTest`.
- **Deploy:** GitHub + Streamlit Community Cloud (one-click, free) for a shareable live link.

6. Why it's useful to a GM / coaching staff

- **It starts the portal window with a shortlist, not a spreadsheet.** Day one of the portal, the staff has a ranked, position-aware board instead of 1,500 names.
- **It's tied to *this* roster's needs.** The board re-ranks the moment you mark a player as leaving — so the night a starter enters the portal, you instantly see who replaces *his* production, not just "good players."
- **It's explainable to non-analysts.** Every score decomposes into production / role / system / attainability with a one-line rationale. That's defensible in a staff meeting and to a head coach who wants the *why*, not a black box.
- **It encodes Illinois's identity from data.** Because system fit is derived from Illinois's actual profile (elite half-court offense, slow tempo, shooting), it won't recommend a player who is good in the abstract but wrong for how Illinois plays.
- **It translates production to *our* level.** The Big Ten projection (calibrated on 990 real transfers) stops a 25-ppg low-major name from masquerading as a 25-ppg Big Ten name — the single most common mistake in reading raw portal stats.

- **It speaks the staff's language.** "Plays like" comps frame an unknown name in terms of players the room already knows, and the 1-page PDF card is something a coach can actually print and carry.
- **It's tunable on the fly.** A coach who wants to prioritize a defensive guard over a scoring one just moves a slider; the board updates in real time.

In practice the staff intersects this board with the live portal list (which they have and which is not a clean public feed): the engine ranks the full pool by fit, and the staff filters to who's actually available. That mirrors how an analytics staffer already works — this just does the heavy ranking in seconds.

7. Honest limitations & next steps

- **Portal availability isn't a clean public dataset**, so the engine ranks the full pool and is meant to be filtered to the live portal list. A natural v2 is to ingest a portal feed and auto-filter.
- **One season of box-score data** underrates injured/young players and can overrate high-usage players on bad teams; attainability, minutes filters, and the Big Ten translation mitigate this but don't eliminate it. Multi-year trends and play-type data (Synergy/Hudl, if licensed) would sharpen it.
- **The translation model is a population average.** It captures how the typical player's box line deflates with the level jump; an individual can beat or miss it, which is why the card surfaces a *level-jump risk* label rather than a false-precision single number.
- **Fit weights are a reasonable default, not gospel** — which is exactly why they're exposed as sliders for the staff to own.
- **No NIL/cost modeling.** A real GM board would layer in budget; that data isn't public.

All data is publicly sourced from BartTorvik. Built as an independent analytics project for the Illinois Men's Basketball Analytics Internship.